

IF25-40401 · DEEP LEARNING · PERTEMUAN 04

DEEP COMPUTER VISION (1): CNN FROM SCRATCH

Konvolusi, Stride, Padding, Pooling, Receptive Field, dan Implementasi PyTorch
LeNet

Tim Dosen Deep Learning · Institut Teknologi Sumatera (ITERA)

Setelah mengikuti pertemuan ini, mahasiswa diharapkan mampu:

- 1 Menjelaskan operasi konvolusi, stride, padding, dan pooling** dalam ekstraksi representasi spasial citra
- 2 Menghitung ukuran feature map dan jumlah parameter konvolusi** secara terstruktur
- 3 Membangun dan melatih CNN dari nol** menggunakan framework PyTorch
- 4 Menganalisis peran weight sharing dan konektivitas lokal** dalam mengatasi keterbatasan MLP

OUTLINE

- 1 Review: Neural Network
- 2 History of CNN
- 3 The CNN Architecture
- 4 Perhitungan Matematis di CNN
- 5 Membangun LeNet Architecture di PyTorch

BAGIAN 01 · REVIEW

Review: Neural Network

Fondasi Deep Learning

Apa itu Neural Network?

Neural Network adalah model komputasi yang terinspirasi dari cara kerja otak manusia, terdiri dari **neuron buatan** yang saling terhubung untuk memproses informasi dan belajar dari data.

Komponen Utama:

- **Input Layer:** Menerima data
- **Hidden Layer:** Memproses informasi
- **Output Layer:** Menghasilkan prediksi
- **Weights & Biases:** Parameter pembelajaran

Fungsi Utama:

- **Pattern Recognition:** Mengenali pola dalam data
- **Classification:** Mengklasifikasi data
- **Regression:** Memprediksi nilai kontinu
- **Feature Learning:** Belajar fitur otomatis

Explore Neural Networks Interactively

Untuk memahami lebih dalam bagaimana **backpropagation** bekerja dalam neural network, kita dapat menggunakan visualisasi interaktif yang memungkinkan eksplorasi real-time dari proses pembelajaran.

Interactive Demo:

<https://xnought.github.io/backprop-explainer/>

Fitur Visualisasi:

- **Real-time computation:** Lihat bagaimana gradient dihitung
- **Interactive parameters:** Ubah weights dan bias secara langsung
- **Visual backpropagation:** Animasi aliran gradient melalui network
- **Step-by-step learning:** Pahami setiap tahap forward dan backward pass

Tahapan Pembelajaran

Neural Network belajar melalui proses **iteratif** yang melibatkan **forward propagation** untuk menghasilkan prediksi dan **backpropagation** untuk memperbarui parameter berdasarkan kesalahan.

Forward Propagation:

- Data input diproses layer demi layer
- Setiap neuron menerapkan **fungsi aktivasi**
- Menghasilkan **output/prediksi** akhir
- Menghitung **loss function**

Backpropagation:

- Menghitung **gradient** dari loss function
- Memperbarui **weights** dan **biases**
- Menggunakan **chain rule** untuk propagasi error
- Optimasi dengan **gradient descent**

Masalah Utama

Neural Network tradisional (**Fully Connected**) memiliki keterbatasan signifikan dalam memproses **data visual** seperti gambar karena tidak mempertimbangkan **struktur spasial** dan **lokalitas** informasi.

Keterbatasan Teknis:

- **Dimensi tinggi:** Gambar $28 \times 28 = 784$ parameter
- **Overfitting:** Terlalu banyak parameter
- **Translation invariance:** Tidak robust terhadap pergeseran
- **Komputasi mahal:** Banyak koneksi tidak perlu

Masalah Struktural:

- Tidak memahami **hierarki fitur**
- Kehilangan informasi **spasial**
- Tidak efisien untuk **pattern lokal**
- Sulit mendeteksi **fitur visual** kompleks

Inovasi CNN

Convolutional Neural Network (CNN) adalah evolusi dari neural network tradisional yang dirancang khusus untuk memproses data visual dengan memanfaatkan **operasi konvolusi** dan **struktur spasial** gambar.

Keunggulan CNN:

- **Local connectivity:** Koneksi lokal, bukan global
- **Parameter sharing:** Filter yang sama untuk seluruh gambar
- **Translation invariance:** Robust terhadap pergeseran
- **Hierarki fitur:** Dari sederhana ke kompleks

Komponen Utama CNN:

- **Convolutional Layer:** Ekstraksi fitur lokal
- **Pooling Layer:** Reduksi dimensi
- **Activation Function:** Non-linearitas
- **Fully Connected Layer:** Klasifikasi akhir

Revolusi dalam Pemrosesan Visual

CNN telah mengubah lanskap **computer vision** dan **artificial intelligence**, memungkinkan terobosan dalam **image recognition**, **object detection**, dan berbagai aplikasi visual lainnya yang sebelumnya tidak mungkin dilakukan.

Aplikasi Real-world:

- **Medical imaging:** Diagnosis kanker, X-ray
- **Autonomous vehicles:** Deteksi objek
- **Face recognition:** Keamanan dan autentikasi
- **Social media:** Auto-tagging, filter

Milestone Penting:

- **ImageNet 2012:** AlexNet mengalahkan metode tradisional
- **Akurasi tinggi:** > 95% pada image classification
- **Transfer learning:** Model pre-trained
- **Deep learning boom:** Memicu era AI modern

Layer Gagal pada Citra?

Masalah Utama Multilayer Perceptron (MLP) pada Citra

Kehilangan Struktur Spasial: Citra di-*flatten* 1D, merusak relasi piksel tetangga.

Ledakan Parameter: Setiap neuron terhubung ke seluruh piksel input.

Perhitungan Parameter MLP:

- Citra CIFAR-10 ($32 \times 32 \times 3 = 3.072$):

$$\text{Bobot} = 3.072 \times 1.024 \approx \mathbf{3,14} \text{ juta}$$

- Resolusi $224 \times 224 \times 3 = 150.528$:

$$\text{Bobot} = 150.528 \times 1.024 \approx \mathbf{154} \text{ juta}$$

- *Hanya untuk layer pertama saja!*

Dampak Negatif:

- **Severe Overfitting:** Kapasitas model berlebihan untuk dataset kecil.
- **Sensitivitas Translasi:** Geser 1 piksel mengubah vektor input sepenuhnya.
- **Beban Komputasi:** Sangat boros memori GPU/RAM.

1. Local Connectivity

Konektivitas Lokal:

- Tiap neuron hanya terhubung ke sub-wilayah kecil (**local receptive field**), misal 3×3 atau 5×5 .
- *Rasional*: Piksel bertetangga memiliki korelasi tinggi (tepi, tekstur, kontur).
- Menjaga topologi spasial 2D citra.

2. Weight Sharing

Pembagian Bobot:

- Kumpulan bobot filter yang sama digeser (*slide*) ke seluruh citra.
- *Rasional (Stationarity)*: Fitur berguna di sudut kiri juga berguna di sudut kanan.
- **Efek**: Filter $5 \times 5 \times 3$ (16 filter) hanya butuh **1.216 bobot** (hemat $> 99.9\%$).

BAGIAN 02 · HISTORI

History of CNN

Evolusi Convolutional Neural Network dari 1960s hingga Era Modern

Inspirasi Biologis (1960s)

Hubel & Wiesel menemukan struktur hierarkis visual cortex kucing yang menjadi blueprint CNN - neuron sederhana mendeteksi tepi, neuron kompleks mendeteksi bentuk yang lebih rumit.

1980 - Neocognitron (Fukushima):

- Leluhur pertama CNN modern
- Memperkenalkan **convolutional layers** (S-cells)
- Menggunakan **pooling layers** (C-cells)
- Keterbatasan: belum ada backpropagation

1989-1998 - LeNet-5 (Yann LeCun):

- CNN praktis pertama dengan **backpropagation**
- Digunakan untuk **pembacaan cek otomatis**
- Struktur: Conv - Pool - Conv - Pool - FC
- Sukses komersial pertama CNN

Periode Sunyi CNN

Selama lebih dari satu dekade, CNN kehilangan popularitas mainstream. Kebutuhan komputasi yang besar dan dominasi metode lain seperti **Support Vector Machines (SVMs)** membuat CNN tertinggal.

Tantangan Era Ini:

- **Keterbatasan komputasi:** CPU tidak cukup kuat
- **Dataset kecil:** Belum ada ImageNet
- **Kompetisi ketat:** SVM lebih efektif
- **Training lambat:** Optimasi belum matang

Persiapan Revolusi:

- **Riset berlanjut:** Perbaikan arsitektur
- **Pengembangan GPU:** NVIDIA CUDA
- **Dataset besar:** Persiapan ImageNet
- **Algoritma baru:** Inovasi training

Menanti Momentum

Periode ini mempersiapkan dua ingredien kunci untuk revolusi CNN: **massive datasets** dan **powerful GPUs**.

2012 - Big Bang: AlexNet

AlexNet menghancurkan kompetisi ImageNet dengan error rate **15.3%** vs **26.2%** (runner-up), memicu revolusi deep learning dan menempatkan CNN di garis depan AI.

Inovasi AlexNet:

- **ReLU activation:** Training lebih cepat
- **Dropout:** Mencegah overfitting
- **GPU training:** Memanfaatkan parallelisasi
- **Deep architecture:** 8 layers

2014-2015 - Arms Race:

- **VGGNet:** Deeper dengan 3x3 conv uniform
- **GoogLeNet:** Inception modules paralel
- **ResNet (2015):** Skip connections, 152+ layers
- **Vanishing gradient:** Diselesaikan ResNet

2016-2020: Refinement era dengan DenseNet, MobileNet (efficiency), EfficientNet (systematic scaling).

Kedatangan Kompetitor Kuat

Vision Transformer (ViT) mengadaptasi arsitektur Transformer dari NLP ke computer vision, mengancam dominasi CNN dengan performa superior pada dataset besar.

2020 - Vision Transformer (ViT):

- Memecah image menjadi **patches**
- Menggunakan **self-attention mechanism**
- Unggul pada dataset besar (JFT-300M)
- Melampaui performa top CNNs

2021-2022 - Dominasi Transformer:

- **Swin Transformer:** Lebih efisien
- **State-of-the-art:** Berbagai benchmark vision
- **Narratif baru:** "Attention is all you need"
- **CNN terancam:** Prediksi akan tergantikan

Momentum Transformer

Komunitas AI mulai mempertanyakan relevansi CNN dengan fixed, local inductive bias vs global attention Transformer.

2017 - "Attention Is All You Need"

Paper revolusioner dari Google Brain yang memperkenalkan **Transformer architecture** - awalnya untuk machine translation, namun tanpa disadari telah meletakkan fondasi untuk **era AI generatif** modern.

Kelahiran GPT Lineage:

- **2018 - GPT-1:** OpenAI adopsi Transformer
- **2019 - GPT-2:** "Too dangerous to release"
- **2020 - GPT-3:** 175B parameters, emergence
- **2022 - ChatGPT:** Mengubah persepsi publik AI

Dominasi Cross-Modal:

- **NLP:** BERT, T5, GPT series
- **Vision:** ViT, DALL-E, Stable Diffusion
- **Multimodal:** GPT-4V, Gemini
- **Foundation models:** Paradigma baru AI

Plot Twist Ironik

Paper yang dimaksudkan untuk machine translation kini mendorong CNN untuk **modernisasi** dan justru memicu **cross-pollination** yang membuat kedua arsitektur saling belajar dan berkembang.

2022 - ConvNeXt: "A ConvNet for the 2020s"

Meta AI membuktikan CNN masih relevan dengan **ConvNeXt** - pure CNN yang mengadopsi design principles Transformer dan mengalahkan Swin Transformer pada ImageNet.

Modernisasi CNN:

- **Large kernel convolutions:** 7x7, bahkan 31x31
- **Training recipes:** Adopsi dari Transformer
- **Block structure:** Inspirasi dari Swin
- **Hybrid models:** CNN + Transformer layers

2023-2025 - Renaissance:

- **Cross-pollination:** Pertukaran ide antar arsitektur
- **State-space models:** Mamba sebagai alternatif
- **Efficiency focus:** CNN masih unggul efisiensi
- **Global receptive field:** CNN menyamai attention

BAGIAN 03 · ARSITEKTUR

The CNN Architecture

Convolutional, Pooling, Fully Connected Layers

CNN Adalah Layer, Bukan Model Lengkap

Seringkali kita mendengar istilah "**Klasifikasi menggunakan CNN**", seolah-olah CNN adalah arsitektur yang berdiri sendiri untuk menyelesaikan tugas klasifikasi. Ini adalah **keliru**.

- **CNN** adalah **layer** yang berfungsi mengekstraksi fitur dari data visual melalui operasi konvolusi.
- Untuk tugas klasifikasi, CNN harus digabungkan dengan **layer lain** seperti **fully connected (FC) layer** di bagian akhir.
- **Arsitektur lengkap** biasanya terdiri dari beberapa convolutional layer, pooling layer, dan diakhiri dengan FC layer untuk menghasilkan output klasifikasi.
- Menyebut "**CNN**" saja tidak cukup—yang benar adalah "**Convolutional Neural Network dengan FC layer untuk klasifikasi**".

Kesimpulan

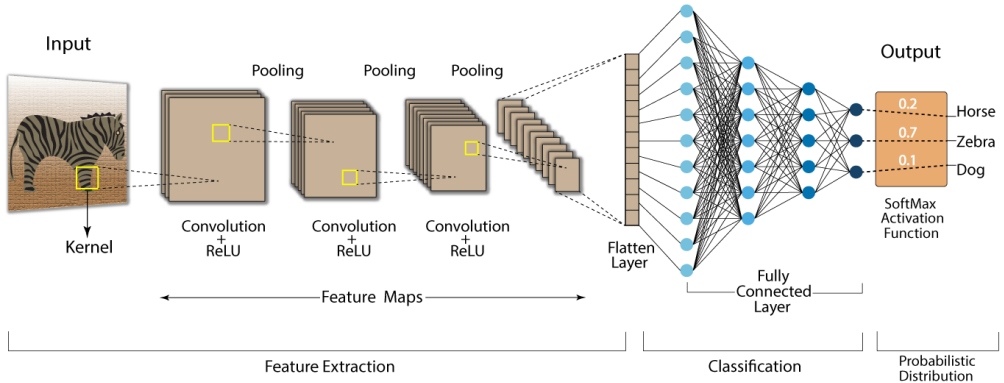
CNN adalah **komponen** dalam arsitektur deep learning, bukan solusi klasifikasi yang berdiri sendiri.

Definisi

Arsitektur model adalah **struktur organisasi** dari layer-layer dan komponen dalam sebuah neural network, termasuk urutan, tipe, dan koneksi antar layer (misal: convolutional, pooling, fully connected). Arsitektur menentukan bagaimana data diproses dari input hingga output, serta bagaimana model belajar dari data.

- Contoh arsitektur: **LeNet**, **AlexNet**, **ResNet**, **ViT**
- Arsitektur dapat disesuaikan untuk berbagai tugas seperti klasifikasi, deteksi objek, segmentasi, dll.
- Pemilihan arsitektur sangat mempengaruhi performa dan efisiensi model deep learning.

Convolution Neural Network (CNN)



Credit: Swapna @ Developers Breach

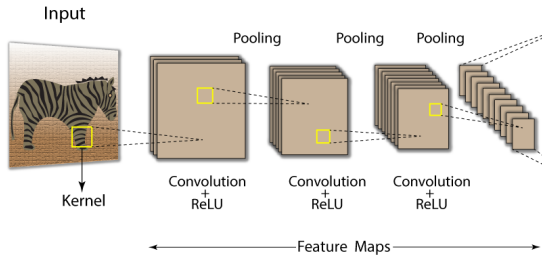
Gambaran Umum

Gambar ini memperlihatkan bagaimana **Convolutional Neural Network (CNN)** mengenali gambar **Zebra**. Prosesnya terdiri dari dua tahap utama: **Ekstraksi Fitur** dan **Klasifikasi**.

- CNN bekerja seperti "otak digital" yang belajar mengenali objek dalam gambar.
- Setiap tahap punya peran khusus, mulai dari mencari ciri-ciri sederhana hingga mengambil keputusan akhir.

Tahap 1: Ekstraksi Fitur

Mencari Ciri-Ciri Penting dalam Gambar



Credit: Swapna @ Developers Breach

Input dan Kernel

Input adalah gambar yang ingin dikenali (misal: zebra), yang bagi komputer hanyalah kumpulan angka (piksel). **Kernel** berfungsi seperti kaca pembesar ajaib yang mencari pola-pola dasar seperti garis atau lengkungan.

Convolution

Kernel melakukan **convolution** dengan menggeser dan memeriksa setiap area gambar, menghasilkan peta ciri untuk setiap pola.

- Kernel bergerak menyisir gambar, mencari pola tertentu.
- Hasilnya adalah **Feature Maps** yang menyorot lokasi pola-pola yang ditemukan.

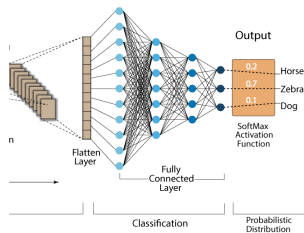
Pooling

Pooling bertugas meringkas peta ciri agar hanya informasi penting yang dipertahankan, membuat proses lebih efisien dan robust.

- Proses convolution dan pooling diulang beberapa kali, sehingga ciri yang dikenali semakin kompleks.

Tahap 2: Klasifikasi

Mengambil Keputusan dari Ciri yang Terkumpul



Credit: Swapna @ Developers Breach

Flatten dan Fully Connected Layer

Semua peta ciri diratakan menjadi satu baris data panjang (**flatten**), lalu diproses oleh **fully connected layer** (otak jaringan) untuk menganalisis dan mengambil keputusan.

- Layer ini menimbang semua ciri yang sudah dikumpulkan.
- Proses berpikir: "Ciri-ciri ini cocok dengan objek apa?"

Softmax Layer

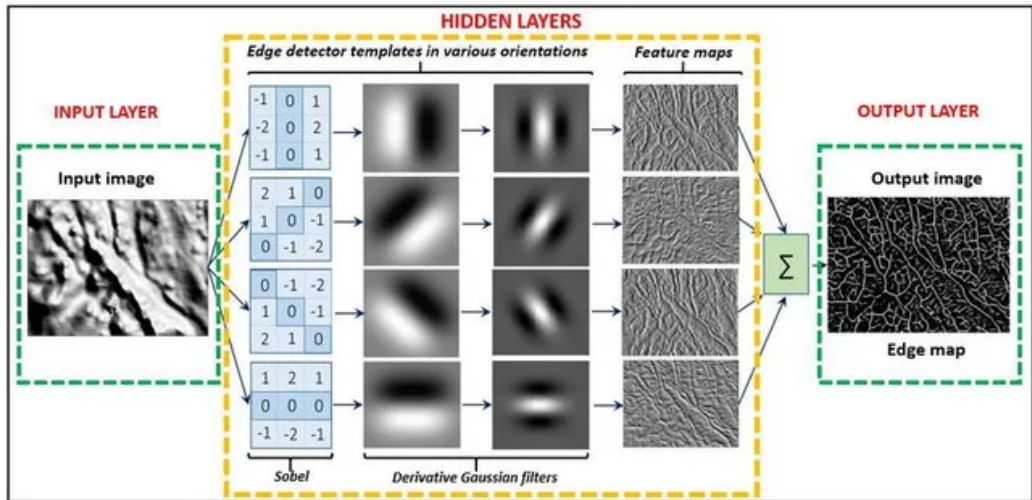
Lapisan terakhir (**Softmax**) memberikan hasil berupa probabilitas untuk setiap kelas objek.

- Misal: 70% yakin gambar adalah Zebra, 20% Kuda, 10% Anjing.
- Probabilitas tertinggi menjadi jawaban akhir sistem.

- 1 **Lihat & Cari Ciri:** Pecah gambar menjadi ciri-ciri dasar (garis, lengkungan).
- 2 **Gabungkan Ciri:** Bentuk ciri kompleks (mata, telinga, pola belang).
- 3 **Simpulkan:** Analisis semua ciri dan tebak objek berdasarkan probabilitas.

Inti CNN

CNN belajar mengenali objek dengan mengekstrak ciri-ciri penting dan mengambil keputusan berdasarkan data yang terkumpul.



Credit: Apurv Chudasama

Representasi Fitur Hierarkis dalam CNN

Dari Piksel Primitif Menuju Semantik Objek

Bagaimana CNN Memahami Dunia Visual?

Lapisan-lapisan konvolusi yang bertumpuk secara hierarkis mengekstrak representasi visual dari tingkat primitif hingga tingkat semantik tinggi:

Layer Awal

Low-Level Features

- Tepi (*edges*)
- Garis vertikal/horizontal
- Gradien warna & kontras
- *Biologis: Sel V1 korteks visual mamalia*

Layer Tengah

Mid-Level Features

- Tekstur (*textures*)
- Sudut (*corners*) & kurva
- Pola geometris berulang
- Motif permukaan objek

Layer Dalam

High-Level Features

- Bagian objek (*parts*): mata, roda, moncong
- Struktur semantik utuh
- Langsung masuk ke fully connected classifier

Tugas Mandiri

Silakan tonton video berikut di rumah untuk memperdalam pemahaman tentang operasi konvolusi dan pooling pada CNN:

- **YouTube:** Convolution and Pooling Explained
- Link: <https://www.youtube.com/watch?v=pj9-rr1wDhM>
- Catat poin-poin penting dan contoh yang diberikan dalam video.
- Siapkan pertanyaan atau hal yang belum dipahami untuk didiskusikan di kelas berikutnya.

BAGIAN 04 · MATEMATIKA

Perhitungan Matematis di CNN

Memahami Operasi Konvolusi dan Pooling secara Matematis

Konsep Fundamental

CNN tidak "melihat" gambar seperti mata manusia. Bagi komputer, gambar adalah **matriks angka** dimana setiap angka merepresentasikan intensitas piksel.

Yang Kita Butuhkan:

- **Input:** Matriks gambar (misal 5x5)
- **Filter/Kernel:** Matriks kecil (misal 3x3)
- **Operasi:** Konvolusi dan Pooling

Tujuan:

- **Konvolusi:** Deteksi pola dan fitur
- **Pooling:** Reduksi dimensi
- **Feature Map:** Representasi fitur

Input dan Filter

Mari kita gunakan contoh sederhana untuk memahami operasi konvolusi.

Input Gambar (5x5):

$$\begin{pmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix}$$

Filter/Kernel (3x3):

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Filter ini mendeteksi pola diagonal (\)

Langkah 1: Element-wise Multiplication

Perkalian Posisi-Posisi yang Sesuai

Posisi Filter di Kiri Atas

Letakkan filter di pojok kiri atas input, lalu kalikan elemen yang bersesuaian.

$$\begin{pmatrix} \frac{1 \times 1}{0 \times 0} & \frac{1 \times 0}{1 \times 1} & \frac{1 \times 0}{1 \times 0} & 0 & 0 \\ \frac{0 \times 0}{0 \times 0} & \frac{0 \times 0}{0 \times 0} & \frac{1 \times 1}{1 \times 1} & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix}$$

Hasil perkalian: 1, 0, 0, 0, 1, 0, 0, 0, 1

Jumlahkan semua: $1 + 0 + 0 + 0 + 1 + 0 + 0 + 0 + 1 = 3$

Angka **3** ini adalah elemen pertama dari feature map!

Langkah 2: Sliding Window (Geser Filter)

Ulangi untuk Seluruh Area

Geser Filter Satu Posisi ke Kanan

Ulangi operasi yang sama untuk posisi filter berikutnya.

$$\begin{pmatrix} 1 & \frac{1 \times 1}{1} & \frac{1 \times 0}{1} & \frac{0 \times 0}{1} & 0 \\ 0 & \frac{1 \times 0}{1} & \frac{1 \times 1}{1} & \frac{1 \times 0}{1} & 0 \\ 0 & \frac{0 \times 0}{1} & \frac{1 \times 0}{1} & \frac{1 \times 1}{1} & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix}$$

Perhitungan: $(1 \times 1) + (1 \times 0) + (0 \times 0) + (1 \times 0) + (1 \times 1) + (1 \times 0) + (0 \times 0) + (0 \times 1) + (1 \times 1) = 3$

- Lanjutkan proses ini untuk seluruh area gambar
- Geser ke kanan, lalu turun satu baris dan ulangi

Feature Map Hasil Konvolusi

Setelah filter bergerak ke seluruh area input, kita mendapatkan feature map berukuran 3x3.

Feature Map:

$$\begin{pmatrix} 3 & 3 & 1 \\ 1 & 4 & 3 \\ 1 & 3 & 3 \end{pmatrix}$$

Interpretasi:

- Nilai besar (seperti **4**) = pola diagonal sangat kuat terdeteksi
- Nilai kecil (seperti **1**) = pola tidak terlalu cocok
- Feature map menunjukkan lokasi dan kekuatan fitur yang dicari

Definisi Stride (S)

Stride menentukan seberapa jauh (berapa piksel) jendela filter melompat pada setiap pergeseran secara horizontal dan vertikal di atas citra input.

Stride = 1 ($S = 1$):

- Filter bergeser piksel demi piksel (*smooth sliding*).
- Ekstraksi fitur sangat padat dan detail.
- Ukuran feature map output hanya menyusut sedikit.
- Standar pada arsitektur konvolusi klasik.

Stride ≥ 2 (misal $S = 2$):

- Filter melompati 1 piksel di setiap langkah.
- Menghasilkan **downsampling spasial** (ukuran feature map berkurang sekitar 50%).
- **Praktik Modern (ResNet/ConvNeXt):** Sering digunakan sebagai pengganti pooling untuk mereduksi dimensi.

Parameter Konvolusi (2): Padding

Menjaga Dimensi Spasial dan Informasi Tepi

Mengapa Perlu Padding (P)?

- **Masalah Penyusutan Spasial:**
Tanpa padding, setiap operasi konvolusi akan mengecilkan ukuran citra. Pada network yang dalam, ukuran akan menjadi 1×1 terlalu cepat!
- **Masalah Informasi Tepi:** Pixel di sudut hanya disentuh kernel satu kali, sedangkan pixel tengah disentuh berkali-kali.

Dua Mode Utama Padding:

- **Valid Padding ($P = 0$):** Tidak ada padding sama sekali. Ukuran output selalu lebih kecil daripada input.
- **Same Padding:** Menambahkan border nol (*Zero-Padding*) sedemikian rupa sehingga ukuran output **sama persis** dengan ukuran input saat $S = 1$:

$$P = \frac{K - 1}{2} \quad (\text{untuk } K \text{ ganjil})$$

Contoh: Kernel $3 \times 3 \implies P = 1$;
Kernel $5 \times 5 \implies P = 2$.

Rumus Matematis Ukuran Feature Map

Menghitung Dimensi Output Konvolusi 2D

Formula Universal Output

Input W_{in} , kernel K , padding P , stride S :

$$W_{out} = \left\lfloor \frac{W_{in} - K + 2P}{S} \right\rfloor + 1$$

- Berlaku identik untuk tinggi H_{out} dan lebar W_{out} .
- Simbol $\lfloor \cdot \rfloor$ adalah pembulatan ke bawah (*floor*).

Contoh Perhitungan:

■ LeNet C1 (Valid):

$$W = 32, K = 5, S = 1, P = 0$$

$$W_{out} = \left\lfloor \frac{32-5+0}{1} \right\rfloor + 1 = \mathbf{28} \implies (28 \times 28)$$

■ Same Conv:

$$W = 32, K = 3, S = 1, P = 1$$

$$W_{out} = \left\lfloor \frac{32-3+2}{1} \right\rfloor + 1 = \mathbf{32} \implies (32 \times 32)$$

■ Stride 2: $W = 32, K = 3, S = 2, P = 1$

$$W_{out} = \left\lfloor \frac{32-3+2}{2} \right\rfloor + 1 = \mathbf{16} \implies (16 \times 16)$$

Model pada Volume 3D

Formula Total Parameter Conv2D

$$\text{Weights} = K_H \times K_W \times C_{in} \times C_{out}, \quad \text{Biases} = C_{out}$$

$$\text{Total Parameter} = (K_H \times K_W \times C_{in} + 1) \times C_{out}$$

Contoh Perhitungan Conv2D (3×3):

- Input RGB ($C_{in} = 3$), $C_{out} = 16$:

$$\text{Params} = (3 \times 3 \times 3 + 1) \times 16 = \mathbf{448}$$

- Input 64 kanal ($C_{in} = 64$), $C_{out} = 128$:

$$\text{Params} = (3 \times 3 \times 64 + 1) \times 128 = \mathbf{73.856}$$

Prinsip Penting

Ukuran spasial citra ($H \times W$) **tidak mempengaruhi** jumlah parameter konvolusi!
Citra 32×32 maupun 1024×1024 memakai jumlah parameter yang sama.

Definisi Receptive Field (RF)

Area spasial pada citra input asli yang berkontribusi terhadap aktivasi suatu neuron pada feature map tertentu.

Pertumbuhan Receptive Field:

- **Layer 1 (3×3):** Melihat area 3×3 citra asli ($RF = 3$).
- **Layer 2 (3×3):** Melihat 3×3 dari Layer 1 $\implies 5 \times 5$ citra asli!
- **Layer 3 (3×3):** Setara melihat 7×7 citra asli.
- *Pooling / Stride 2 mempercepat pertumbuhan RF.*

Insight CS231n & VGGNet

Dua filter 3×3 vs satu filter 5×5 :

- Keduanya memiliki RF efektif 5×5 .
- Bobot $1 \times (5 \times 5) = 25$ parameter.
- Bobot $2 \times (3 \times 3) = 18$ parameter (hemat 28%!).
- Memiliki **dua non-linearitas (ReLU)**.

Tujuan Pooling

Pooling berfungsi untuk **mengurangi ukuran** feature map sambil **mempertahankan informasi penting**. Ini membuat model lebih efisien dan robust.

Yang Dibutuhkan:

- Feature map hasil konvolusi
- Ukuran window (biasanya 2x2)
- Jenis pooling (Max atau Average)

Keuntungan:

- Reduksi parameter
- Mengurangi overfitting
- Translation invariance
- Komputasi lebih cepat

Max Pooling: "Si Paling Menonjol"

Ambil Nilai Maksimum dari Setiap Blok

Contoh Feature Map (4x4)

Mari kita aplikasikan max pooling pada feature map berikut:

$$\begin{pmatrix} 3 & 3 & 2 & 1 \\ 1 & 4 & 3 & 0 \\ 1 & 2 & 3 & 3 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

Bagi menjadi blok 2x2:

$$\begin{pmatrix} 3 & 3 \\ 1 & 4 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 3 & 0 \end{pmatrix}$$
$$\begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 3 & 3 \\ 1 & 0 \end{pmatrix}$$

Max dari setiap blok: Biru=4, Merah=3, Hijau=2, Oranye=3

Average Pooling: "Si Rata-Rata"

Ambil Nilai Rata-Rata dari Setiap Blok

Menggunakan Feature Map yang Sama

Kali ini kita hitung rata-rata dari setiap blok 2x2.

Perhitungan untuk setiap blok:

- **Blok biru:** $(3 + 3 + 1 + 4) \div 4 = 11 \div 4 = 2.75$
- **Blok merah:** $(2 + 1 + 3 + 0) \div 4 = 6 \div 4 = 1.5$
- **Blok hijau:** $(1 + 2 + 0 + 1) \div 4 = 4 \div 4 = 1$
- **Blok oranye:** $(3 + 3 + 1 + 0) \div 4 = 7 \div 4 = 1.75$

Hasil Max Pooling:

$$\begin{pmatrix} 4 & 3 \\ 2 & 3 \end{pmatrix}$$

Hasil Average Pooling:

$$\begin{pmatrix} 2.75 & 1.5 \\ 1 & 1.75 \end{pmatrix}$$

Perbandingan Max vs Average Pooling

Kapan Menggunakan Yang Mana?

Max Pooling:

- Lebih **populer** dalam praktik
- Mempertahankan **fitur terkuat**
- Baik untuk deteksi fitur yang **spars**
- Memberikan **invariance** yang lebih baik

Kapan digunakan:

- Image classification
- Object detection
- Ketika fitur yang prominent penting

Average Pooling:

- Mempertahankan informasi **keseluruhan**
- Lebih **smooth** dalam transisi
- Mengurangi **noise**
- Baik untuk texture analysis

Kapan digunakan:

- Global Average Pooling
- Ketika detail halus penting
- Saat ingin menghindari overfitting

Krusial Mengapa CNN Sangat Robust

1. Invariansi Translasi Lokal

Robust terhadap Pergeseran Objek:

- Jika objek atau fitur bergeser sedikit (1–2 piksel), aktivasi fitur hanya bergeser di dalam jendela pooling 2×2 .
- Nilai **maksimum yang diteruskan tetap sama**:

$$\max(0, 8, 1, 0) = \max(8, 0, 0, 1) = 8$$

- CNN fokus pada "*apakah fitur ada*", bukan posisi presisi sub-pikselnya.

2. Reduksi Dimensi Spasial

Efisiensi Komputasi & Memori:

- Pooling 2×2 stride 2 memotong 75% jumlah piksel ($H/2 \times W/2$).
- Mengurangi beban memori dan komputasi (FLOPs) secara drastis pada layer berikutnya.
- Memperbesar **effective receptive field** neuron di layer selanjutnya.

Ringkasan Proses

CNN bekerja melalui serangkaian operasi matematis sederhana yang diulang berkali-kali untuk mengekstrak dan meringkas fitur.

1 Konvolusi:

- Sliding window dengan element-wise multiplication
- Menghasilkan feature map yang menunjukkan lokasi fitur

2 Pooling:

- Reduksi dimensi dengan max atau average
- Mempertahankan informasi penting sambil mengurangi kompleksitas

3 Iterasi:

- Proses ini diulang dengan berbagai filter
- Setiap layer mendeteksi fitur yang semakin kompleks

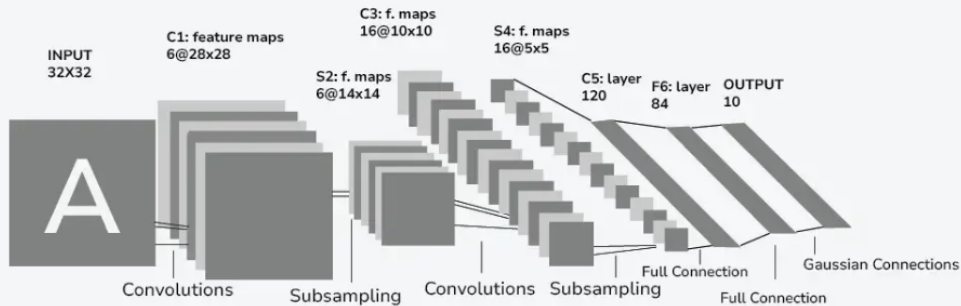
BAGIAN 05 · IMPLEMENTASI

Membangun LeNet Architecture di PyTorch

Implementasi CNN dari Awal menggunakan PyTorch



LeNet Architecture



Credit: geeksforgeeks

Yann LeCun: Bapak CNN Modern

Pelopor Deep Learning dan LeNet



Credit: Bloomberg via Getty Images

Yann LeCun adalah ilmuwan komputer asal Prancis yang dikenal sebagai pelopor **Convolutional Neural Network (CNN)**. Ia menciptakan arsitektur **LeNet** pada akhir 1980-an, yang menjadi fondasi utama perkembangan deep learning modern, khususnya untuk pengenalan gambar dan tulisan tangan. LeCun juga menerima **Turing Award 2018** bersama Geoffrey Hinton dan Yoshua Bengio atas kontribusinya di bidang AI.

LeNet-5 (Yann LeCun, 1998)

LeNet-5 adalah arsitektur CNN klasik yang terdiri dari 7 lapisan utama (tidak termasuk input), dirancang untuk mengenali angka tulisan tangan (MNIST).

- Input: Gambar 32x32 piksel (MNIST dipadding dari 28x28)
- Lapisan: Konvolusi, Pooling, Fully Connected
- Output: 10 kelas (digit 0-9)

C1 - Convolutional Layer

- 6 filter (kernel) berukuran 5x5
- Menyapu seluruh gambar input (32x32)
- Menghasilkan 6 **feature maps** berukuran 28x28
- Mendeteksi fitur sederhana: tepi, garis, sudut

S2 - Pooling Layer

- Pooling (biasanya average pooling) 2x2
- Mengurangi ukuran feature map dari 28x28 menjadi 14x14
- Jumlah feature map tetap: 6
- Membuat model lebih efisien dan robust terhadap pergeseran

C3 - Convolutional Layer

- 16 filter 5x5 pada hasil S2
- Output: 16 feature maps berukuran 10x10
- Mendeteksi fitur lebih kompleks (kombinasi fitur dasar)

S4 - Pooling Layer

- Pooling 2x2 pada hasil C3
- Output: 16 feature maps berukuran 5x5
- Reduksi dimensi, tetap 16 maps

C5 - Convolutional Layer

- 120 filter 5x5 pada hasil S4 (input 5x5)
- Setiap filter menghasilkan 1 nilai (output 1x1)
- Output: 120 neuron (vektor 1D)

F6 - Fully Connected Layer

- 120 neuron dari C5 terhubung ke 84 neuron
- Lapisan ini melakukan proses klasifikasi tingkat tinggi

Output Layer

- 84 neuron F6 terhubung ke 10 neuron output
- Setiap neuron mewakili satu kelas (digit 0-9)
- Fungsi aktivasi Softmax menghasilkan probabilitas tiap kelas
- Prediksi akhir: kelas dengan probabilitas tertinggi

Tujuan

Kita akan mengimplementasikan arsitektur LeNet-5 menggunakan PyTorch dan melihat struktur detail setiap layer dengan **torchinfo**.

Yang akan kita pelajari:

- Definisi kelas LeNet5 dengan PyTorch
- Struktur feature extractor dan classifier
- Penggunaan torchinfo untuk analisis model
- Memahami flow data melalui setiap layer

Prasyarat

Pastikan sudah menginstall: `pip install torch torchinfo`

Import Essential Libraries

```
import torch
import torch.nn as nn
from torchinfo import summary
```

Penjelasan Library:

- `torch`: Core PyTorch library untuk tensor operations
- `torch.nn`: Modul untuk neural network layers dan functions
- `torchinfo`: Library untuk menampilkan summary model yang detail

Definisi Kelas LeNet5: Constructor

Inisialisasi Layer-layer CNN

```
class LeNet5(nn.Module):
    """
    Implementasi arsitektur LeNet-5 (1998) menggunakan PyTorch.
    Arsitektur ini didesain untuk input gambar grayscale 32x32 piksel.
    """
    def __init__(self):
        super(LeNet5, self).__init__()

        # === 1. BAGIAN EKSTRAKTOR FITUR ===
        self.feature_extractor = nn.Sequential(
            # Lapisan C1: Konvolusi pertama
            nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, stride=1),
            nn.Tanh(),

            # Lapisan S2: Subsampling (Average Pooling)
            nn.AvgPool2d(kernel_size=2, stride=2),

            # Lapisan C3: Konvolusi kedua
            nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1),
            nn.Tanh(),

            # Lapisan S4: Subsampling (Average Pooling)
            nn.AvgPool2d(kernel_size=2, stride=2)
        )
```

```
# === 2. BAGIAN KLASIFIKATOR ===
self.classifier = nn.Sequential(
    # Lapisan C5 (sebagai Fully Connected pertama)
    nn.Linear(in_features=16 * 5 * 5, out_features=120),
    nn.Tanh(),

    # Lapisan F6 (Fully Connected kedua)
    nn.Linear(in_features=120, out_features=84),
    nn.Tanh(),

    # Lapisan Output (10 kelas untuk digit 0-9)
    nn.Linear(in_features=84, out_features=10)
)
```

Perhitungan input size untuk Linear layer:

- Setelah S4: feature maps berukuran $16 \times 5 \times 5$
- Flatten: $16 \times 5 \times 5 = 400$ features
- Input ke Linear pertama: $400 \rightarrow 120 \rightarrow 84 \rightarrow 10$

```
def forward(self, x):  
    """  
    Mendefinisikan alur maju (forward pass) dari data melalui model.  
    """  
    # Lewatkan input melalui ekstraktor fitur  
    x = self.feature_extractor(x)  
  
    # Flatten output dari 2D menjadi 1D sebelum masuk ke classifier  
    # Dimensi 0 (batch) dipertahankan  
    x = torch.flatten(x, 1)  
  
    # Lewatkan data yang sudah diratakan melalui classifier  
    logits = self.classifier(x)  
    return logits
```

Penjelasan forward pass:

- Feature extractor: Conv → Pool → Conv → Pool
- Flatten: Ubah tensor 4D menjadi 2D untuk FC layers
- Classifier: FC → FC → FC (output logits)

```
if __name__ == '__main__':  
    # Membuat instance dari model LeNet5  
    model = LeNet5()  
  
    # Menentukan ukuran input untuk summary  
    # Format: (batch_size, channels, height, width)  
    input_size = (1, 1, 32, 32)  
  
    # Menampilkan ringkasan model menggunakan torchinfo  
    print("=" * 65)  
    print("          Ringkasan Arsitektur LeNet-5")  
    print("=" * 65)  
    summary(model, input_size=input_size)
```

Input size explanation:

- Batch size: 1 (untuk testing)
- Channels: 1 (grayscale image)
- Height x Width: 32 x 32 (sesuai paper LeNet-5)

Transformasi Ukuran Data

Mari kita trace bagaimana ukuran data berubah melalui setiap layer:

- 1 Input:** [1, 1, 32, 32] - Batch=1, Channel=1, 32x32 pixels
- 2 Conv2d C1:** [1, 6, 28, 28] - 6 feature maps, 28x28 each
- 3 AvgPool S2:** [1, 6, 14, 14] - Ukuran dipotong setengah
- 4 Conv2d C3:** [1, 16, 10, 10] - 16 feature maps, 10x10 each
- 5 AvgPool S4:** [1, 16, 5, 5] - Ukuran dipotong setengah lagi
- 6 Flatten:** [1, 400] - $16 \times 5 \times 5 = 400$ features
- 7 Linear F6:** [1, 120] $\rightarrow$ [1, 84] $\rightarrow$ [1, 10]

Insight: Dimensi spasial terus mengecil, jumlah channel bertambah.

Total Parameter: 61,706

Mari kita breakdown jumlah parameter di setiap layer:

- **Conv2d C1:** 156 params = $(5 \times 5 \times 1 + 1) \times 6 = 26 \times 6$
- **Conv2d C3:** 2,416 params = $(5 \times 5 \times 6 + 1) \times 16 = 151 \times 16$
- **Linear C5:** 48,120 params = $(400 + 1) \times 120 = 401 \times 120$
- **Linear F6:** 10,164 params = $(120 + 1) \times 84 = 121 \times 84$
- **Linear Output:** 850 params = $(84 + 1) \times 10 = 85 \times 10$

Observasi Penting

80% parameter ada di fully connected layers! Ini menunjukkan mengapa CNN lebih efisien daripada fully connected network biasa.

Keunggulan nn.Sequential:

- **Clean code:** Struktur yang rapi
- **Easy debugging:** Layer terorganisir
- **Modular:** Mudah dimodifikasi
- **Automatic forward:** PyTorch handle flow

Benefit Torchinfo:

- **Visual debugging:** Lihat shape setiap layer
- **Parameter counting:** Hitung kompleksitas
- **Memory analysis:** Estimasi RAM usage
- **Architecture validation:** Cek implementasi

Best Practice

Selalu gunakan `torchinfo.summary()` setelah membuat model untuk memverifikasi arsitektur dan mendeteksi potential issues sebelum training.

Tiga Komponen Esensial Pelatihan CNN

Untuk melatih model CNN dari awal, kita mendefinisikan model, fungsi loss, dan optimizer:

```
model = LeNet5()                                # Inisialisasi model
criterion = nn.CrossEntropyLoss()               # Loss multi-kelas
optimizer = torch.optim.Adam(model.parameters(), lr=0.001) # Optimizer (atau SGD)
```

Fungsi Komponen:

- **CrossEntropyLoss**: Gabungan LogSoftmax + NLLLoss untuk klasifikasi.
- **Adam / SGD**: Algoritma pembaruan bobot berbasis gradien.

DataLoader:

- Membagi dataset menjadi *mini-batches* (misal 32 atau 64 citra).
- Menyediakan *shuffle* otomatis pada setiap epoch.

Training Loop CNN di PyTorch

Alur 5 Langkah Wajib dalam Setiap Batch

```
model.train() # Mode training
for epoch in range(num_epochs):
    for images, labels in train_loader:
        optimizer.zero_grad()           # 1. Reset gradien akumulasi
        outputs = model(images)          # 2. Forward pass: hitung output
        loss = criterion(outputs, labels) # 3. Hitung loss
        loss.backward()                  # 4. Backward: hitung gradien dL/dw
        optimizer.step()                 # 5. Update bobot ( $w = w - lr * grad$ )
```

Penting: Mengapa `zero_grad()` Wajib?

Secara default di PyTorch, gradien di-**akumulasi** (*accumulated*). Jika tidak di-reset di awal setiap batch, gradien akan terus bertambah dan merusak arah pembaruan bobot model!

Evaluasi Performa Model (Inference Mode) Mengukur Akurasi pada Data Uji (Test Dataset)

```
model.eval() # Mode evaluasi (nonaktifkan behavior training Dropout/BatchNorm)
correct, total = 0, 0

with torch.no_grad(): # Matikan autograd (hemat memori GPU & percepat inferensi)
    for images, labels in test_loader:
        outputs = model(images)
        _, predicted = torch.max(outputs, 1) # Prediksi kelas probabilitas tertinggi
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f"Akurasi Test Set: {100 * correct / total:.2f}%")
```

Mengapa `torch.no_grad()` Penting Saat Evaluasi?

Evaluasi tidak membutuhkan turunan/gradien. `torch.no_grad()` mematikan graf komputasi autograd, membebaskan memori RAM/GPU, dan mempercepat inferensi.

- 1 Operasi Spasial Citra (Sub-CPMK 1):** Konvolusi mengekstrak representasi spasial; **Stride** (S) langkah geser; **Padding** (P) menjaga border; **Pooling** mereduksi dimensi & memberi invariansi translasi.
- 2 Aritmetika CNN (Sub-CPMK 2):** Output: $O = \lfloor \frac{W-K+2P}{S} \rfloor + 1$. Parameter: $(K \times K \times C_{in} + 1) \times C_{out}$ (mandiri terhadap resolusi spasial citra input).
- 3 Pilar Efisiensi vs MLP (Sub-CPMK 4):** **Local connectivity** + **Weight sharing** menyelesaikan ledakan jutaan bobot MLP dan menjaga topologi 2D citra.
- 4 Implementasi PyTorch (Sub-CPMK 3):** LeNet-5 via `nn.Sequential`, verifikasi bentuk via `torchinfo`, dan alur 5 langkah training loop.

Persiapan Minggu Depan (Pertemuan 5)

Deep Computer Vision (2): CNN dan Variasinya — Evolusi modern: AlexNet, VGG, ResNet (residual skip connections), MobileNet, dan ConvNeXt.

IF25-40401 · DEEP LEARNING

Terima Kasih

Pertemuan Berikutnya: Deep Computer Vision (2) — CNN dan Variasinya

Tim Dosen Deep Learning · Institut Teknologi Sumatera (ITERA)